```
    fill_in 'Address', with: '123 Main Street'
    fill_in 'Email', with: 'dave@example.com'

    select 'Check', from: 'Pay type'
    fill_in "Routing number", with: "123456"
    fill_in "Account number", with: "987654"

    click_button "Place Order"
    assert_text 'Thank you for your order'

    perform_enqueued_jobs
    perform_enqueued_jobs
    assert_performed_jobs 2

    orders = Order.all
    assert_equal 1, orders.size

    order = orders.first
    assert_equal "Dave Thomas",      order.name
    assert_equal "123 Main Street",  order.address
    assert_equal "dave@example.com", order.email
    assert_equal "Check",            order.pay_type
    assert_equal 1, order.line_items.size

    mail = ActionMailer::Base.deliveries.last
    assert_equal ["dave@example.com"],                mail.to
    assert_equal 'Sam Ruby <depot@example.com>',      mail[:from].value
    assert_equal "Pragmatic Store Order Confirmation", mail.subject
  end
```

This test reads almost like English. Since you now need to submit the form and assert that an order was created, you start by clearing out any orders in the test database that might be hanging around from previous test runs.

Next, you add an item to the cart, check out and fill in the pay type details, place your order, and verify that you get a *Thank you* response.

Since this test is about the user's experience end-to-end, you don't need to look at the jobs that have been queued—instead we need to make sure they are executed. It's sufficient to assert the *results* of those jobs having been executed. To that end, the method `perform_enqueued_jobs` will perform any jobs that get enqueued inside the block of code given to it.Since our `ChangeOrderJob` enqueues